Intern Management System — AI Agent Prompt Pack

> Senior Developer Brief: Bug Fixes & Quality-of-Life Improvements Hand this file to an AI coding agent. Work through each **PROMPT** section one at a time.

PROMPT 0 — PROJECT CONTEXT (Read First, Always Reference)

You are a senior full-stack developer working on an **Intern Management System** built with:

- **Framework:** Next.js 14 (App Router), TypeScript, Tailwind CSS
- **Auth:** NextAuth.js v4, JWT strategy, roles: `admin`, `mentor`, `intern`
- **Backend/DB:** Hasura GraphQL (admin-secret auth) → PostgreSQL 14
- **State:** Redux Toolkit (`notificationSlice`, `uiSlice`, `authSlice`, `loadingSlice`)
- **UI libs:** SweetAlert2 (toasts/alerts), Recharts (charts), Framer Motion, Lucide icons
- **Email:** Custom `emailService.ts` with provider abstraction
- **OTP / Password reset:** `password_reset_tokens` table, existing routes at `/api/auth/forgot-password`, `/api/auth/verify-otp`, `/api/auth/reset-password`

Project file structure (important paths)

```
src/
├── app/
│   ├── (admin)/dashboard/admin/
│   │   ├── attendance/page.tsx
│   │   ├── import/page.tsx          ← BulkImport lives here
│   │   ├── interns/page.tsx         ← Main interns CRUD table
│   │   ├── mentors/page.tsx         ← Main mentors CRUD table
│   │   ├── reports/page.tsx
│   │   ├── tasks/page.tsx
│   │   ├── logs/page.tsx
│   │   └── page.tsx                 ← Admin dashboard (stats + charts)
│   ├── (mentor)/dashboard/mentor/
│   │   ├── interns/page.tsx
│   │   ├── reports/page.tsx
│   │   └── tasks/page.tsx
```

```
│   ├── (intern)/dashboard/intern/
│   │   ├── tasks/page.tsx
│   │   ├── reports/page.tsx
│   │   ├── submit-report/page.tsx
│   │   └── attendance/page.tsx
│   ├── (shared)/dashboard/calendar/page.tsx
│   ├── access-denied/page.tsx
│   ├── auth/login/page.tsx
│   ├── auth/forgot-password/page.tsx
│   ├── auth/verify-otp/page.tsx
│   ├── auth/reset-password/page.tsx
│   ├── profile/page.tsx
│   └── api/
│       ├── auth/[...nextauth]/route.ts
│       ├── auth/users/route.ts
│       ├── admin/import/route.ts
│       ├── admin/stats/route.ts
│       ├── interns/route.ts  +  [id]/route.ts
│       ├── mentors/route.ts  +  [id]/route.ts
│       ├── tasks/route.ts    +  [id]/route.ts
│       ├── reports/route.ts  +  [id]/route.ts
│       ├── attendance/route.ts
│       ├── events/route.ts   +  [id]/route.ts
│       ├── activity/route.ts
│       └── profile/route.ts
├── components/
│   ├── features/
│   │   ├── NotificationBell.tsx     ← Currently uses MOCK data only
│   │   ├── BulkImportForm.tsx
│   │   ├── PerformanceChart.tsx     ← Uses Recharts
│   │   ├── AttendanceCard.tsx
│   │   ├── KanbanBoard.tsx
│   │   └── ...
│   ├── layout/
│   │   ├── Sidebar.tsx
│   │   ├── DashboardLayout.tsx
│   │   ├── DashboardHeader.tsx
│   │   └── RoleGuard.tsx            ← Client-side only role guard
│   └── ui/
│       ├── AccessDenied.tsx
│       ├── Modal.tsx
│       ├── Input.tsx
│       ├── Select.tsx
│       └── ...
```

```
├── lib/
│   ├── auth.ts                    ← NextAuth config + getServerSession
│   ├── db.ts                      ← Hasura query wrappers + row mappers
│   ├── hasura.ts                  ← Core hasuraCall() fetch wrapper
│   ├── constants.ts               ← DEPARTMENTS[], USER_ROLES, etc. (hardcoded)
│   ├── notifications.ts           ← SweetAlert2 toast/confirm helpers
│   ├── graphql/
│   │   ├── queries.ts
│   │   └── mutations.ts
│   ├── email/emailService.ts
│   ├── redux/slices/notificationSlice.ts
│   └── ...
```

Database schema summary (schema_v2.sql)

Key tables: `departments`, `users`, `profiles`, `interns`, `tasks`, `task_assignments`, `reports`, `attendance`, `events`, `activity_logs`, `password_reset_tokens`.

- `departments.head_id` → FK to `users(id)` — **exists in DB but has no UI**
- `interns` table has: `college_name`, `university`, `start_date`, `end_date`, `status`, `mentor_id`
- `interns` does **NOT** yet have: `graduation_degree` field — **needs a schema migration**
- `users` does **NOT** yet have: `recovery_email` field — **needs a schema migration**

Critical existing gaps (do not re-introduce these):

1. **No** `middleware.ts` — zero server-side route protection today
2. `NotificationBell` — 100% mock data, not wired to real DB events
3. **DEPARTMENTS** is hardcoded in **three places**: `constants.ts`, `admin/import/route.ts`, `(admin)/dashboard/admin/interns/page.tsx` — must be made dynamic
4. **No Formik/Yup** anywhere — forms use manual `useState` for errors
5. **Email change** in `profile/page.tsx` has no OTP verification
6. `RoleGuard` is client-side only — can be bypassed by direct URL navigation
7. **No export modal** — only a basic `downloadCSV()` utility exists
8. **BulkImport result** shows only `{ success: N, failed: N, errors: string[] }` — no downloadable error file
9. **SweetAlert2** `showAlert`/`showToast` background colors are hardcoded white — dark mode unaware

House rules for every change

- **Never break existing API contracts** — keep the same endpoint paths and response shapes, only add to them
- **Always use** `auth()` **from** `@/lib/auth` for server-side session checks in API routes
- **All new DB columns** must be added via `ALTER TABLE` SQL that you include in your answer
- **Keep DEPARTMENTS dynamic** from the database after Prompt 4 is complete — never hardcode them again
- **Scope:** This is an intern management system. Do not introduce unrelated features.

PROMPT 1 — Middleware & Role-Based Route Protection

What to build

Create `src/middleware.ts` at the project root (Next.js App Router convention). This is the **only place** server-side route protection should live.

Requirements

1. **Protected route map** — define a constant that maps URL prefixes to allowed roles:

```
/dashboard/admin/*    → ['admin']
/dashboard/mentor/*   → ['mentor']
/dashboard/intern/*   → ['intern']
/dashboard/calendar   → ['admin', 'mentor', 'intern']  (shared)
/profile              → ['admin', 'mentor', 'intern']
/api/admin/*          → ['admin']
/api/interns/*        → ['admin', 'mentor']
/api/mentors/*        → ['admin']
/api/tasks/*          → ['admin', 'mentor', 'intern']
/api/reports/*        → ['admin', 'mentor', 'intern']
/api/attendance/*     → ['admin', 'mentor', 'intern']
/api/events/*         → ['admin', 'mentor', 'intern']
/api/profile          → ['admin', 'mentor', 'intern']
/api/activity         → ['admin']
```

2. **Flow:**
   - If the user is not authenticated and tries to access any protected route → redirect to `/auth/login`
   - If the user IS authenticated but their role is not in the allowed list for that route → redirect to `/access-denied`
   - Public routes (`/auth/*`, `/access-denied`, `/api/auth/*`) must pass through unconditionally
   - Use `getToken` from `next-auth/jwt` (not `getServerSession` — middleware runs on the Edge)
3. **The `access-denied` page** (`src/app/access-denied/page.tsx`) already uses `<AccessDenied />`. Enhance `src/components/ui/AccessDenied.tsx` to accept an optional `requiredRole` prop and display a clean, role-aware message, e.g.: *"This page is for Admins only. You are logged in as Intern."* Show the user's actual role from the session. Include a "Go to my Dashboard" button that routes them to their correct dashboard based on their role.
4. **`matcher` config** in the middleware must cover:

```ts
matcher: ['/((?!_next/static|_next/image|favicon.ico|public/).*)']
```

5. **Keep `RoleGuard`** component in place — it provides a second layer of protection and graceful loading UI. Do NOT remove it.
6. **API routes:** For API routes, when access is denied, return `NextResponse.json({ error: 'Forbidden' }, { status: 403 })` — do NOT redirect.

PROMPT 2 — Formik + Yup Validation (All Forms)

What to build

Install Formik and Yup. Replace all manual `useState`-based form validation with Formik + Yup schemas. This applies to every form that creates or edits data.

Forms to convert (in priority order)

A. Intern creation/edit form — `src/app/(admin)/dashboard/admin/interns/page.tsx`

Fields and rules:

- `name`: required, min 2 chars, max 100 chars, no numbers
- `email`: required, valid email format
- `phone`: optional, if provided must match `^\+?[0-9\s\-()]{7,15}$`
- `department`: required, must be one of the dynamic department list
- `mentorId`: required (select from loaded mentors)
- `startDate`: required, must not be in the past (warn, not block), must be a valid date
- `endDate`: optional, if provided must be >= `startDate`
- `collegeName`: optional, max 150 chars
- `university`: optional, max 150 chars
- `graduationDegree`: optional, must be one of: `B.Tech`, `B.E.`, `MCA`, `BCA`, `M.Tech`, `MBA`, `B.Sc`, `M.Sc`, `Other`

B. Mentor creation/edit form — `src/app/(admin)/dashboard/admin/mentors/page.tsx`

- `name`: required, min 2, max 100 chars
- `email`: required, valid email
- `phone`: optional, same regex as above
- `department`: required

C. Task creation/edit form — tasks pages (admin + mentor)

- `title`: required, min 3 chars, max 200 chars
- `description`: optional, max 2000 chars
- `deadline`: optional, if provided must be today or future
- `priority`: required, enum `low | medium | high`
- `assignedTo` (intern IDs): required unless `assignedToAll` is checked

D. Report submission form — `src/app/(intern)/dashboard/intern/submit-report/page.tsx`

- `workDescription`: required, min 20 chars, max 3000 chars
- `hoursWorked`: required, numeric, between 0.5 and 12

- `reportDate`: required, cannot be a future date

E. Profile / email change — `src/app/profile/page.tsx`

- `name`: required, min 2 chars
- `email`: required, valid email format, must differ from current email

F. Password change form — `src/components/features/ChangePasswordModal.tsx`

- `currentPassword`: required
- `newPassword`: required, min 8 chars, must contain at least 1 uppercase, 1 lowercase, 1 digit, 1 special char (`@$!%*?&`)
- `confirmPassword`: required, must match `newPassword`

UI requirements for error display

1. **Field-level**: Show the error message in red text directly beneath the field immediately on `onBlur` (touched fields). Use the existing `Input` component's potential `error` prop — add one if it doesn't exist.
2. **Submit-level**: If the form has errors on submit attempt, show a compact red summary banner at the top of the form: *"Please fix the errors below before submitting."*
3. **Do NOT use alert boxes or SweetAlert for validation errors** — keep them inline.
4. **Loading states**: Disable the submit button and show a spinner while submitting.

Implementation notes

- Install: `npm install formik yup`
- Create a shared `src/lib/validations/schemas.ts` file that exports all Yup schemas, so they can be reused across pages and in API-side validation too
- Keep the existing `showToast` calls for **server-side success/error responses** — Formik handles client-side field errors only

---

PROMPT 3 — Password Breach Checking

What to build

Integrate the **Have I Been Pwned (HIBP) Pwned Passwords API** into the password change flow. This is an industry-standard, privacy-safe API.

How HIBP works (k-Anonymity model — implement exactly this)

1. Take the new password the user is trying to set
2. Compute its SHA-1 hash (hex, uppercase)
3. Send only the **first 5 characters** of the hash to `https://api.pwnedpasswords.com/range/{first5chars}`
4. The API returns a list of `SUFFIX:COUNT` pairs — all hashes that start with those 5 chars
5. Check if the **remaining 35 chars** of your hash appear in the response
6. If found with a count > 0 → the password has appeared in breaches

The plaintext password is NEVER sent to any external API.

Where to integrate

1. **`src/lib/validations/hibp.ts`** — create a utility:

```ts
export async function isPasswordPwned(password: string): Promise<number>
// Returns the number of times this password appears in breaches. 0 = safe.
```

Implement using the Node.js `crypto` module (`createHash('sha1')`). Handle network failures gracefully — if HIBP is unreachable, log a warning and return 0 (don't block the user).

2. **In the Formik Yup schema for `newPassword`** (from Prompt 2F): Add an async Yup test:

```ts
```

```
  .test('not-pwned', 'This password has appeared in data breaches. Please choose a
    if (!value || value.length < 8) return true; // let required/min handle it
    const count = await isPasswordPwned(value);
    return count === 0;
  })
```

3. **Show a specific UX message** when a password is breached: *"This password was found in known data breach lists. Please choose a unique password."* — shown inline under the new password field, not as an alert.
4. **Also run the check server-side** in `src/app/api/auth/users/route.ts` and `src/app/api/profile/route.ts` (password change endpoint) as a secondary defense. If the count is > 100 (clearly compromised), reject with HTTP 422 and message: `{ error: "PASSWORD_BREACHED", message: "..." }`.

PROMPT 4 — Dynamic Department Management

What to build

Departments are currently **hardcoded** in three places. Make them dynamic (loaded from the `departments` table) and add a Department Management UI for admins.

Part A — New API route: `/api/departments`

Create `src/app/api/departments/route.ts`:

- **GET**: Return all departments with `id`, `name`, and the `head` user details (if set). No auth required (used in forms).
- **POST** (admin only): Create a new department. Validate: `name` required, max 60 chars, strip extra whitespace, check for duplicate (case-insensitive) before inserting — return HTTP 409 if duplicate.
- **PATCH** `/api/departments/[id]`: Update `name` or `head_id` (admin only).
- **DELETE** `/api/departments/[id]` (admin only): Block deletion if the department still has users assigned to it — return HTTP 409 with a clear message.

GraphQL mutations/queries to add in `src/lib/graphql/`:

- `GET_ALL_DEPARTMENTS` — include `head { id profile { name } }`
- `CREATE_DEPARTMENT`
- `UPDATE_DEPARTMENT`
- `DELETE_DEPARTMENT` — check for existing users first

Part B — Remove all hardcoded DEPARTMENTS

- In `src/lib/constants.ts`: Keep the type definition but remove the hardcoded array (or mark it as legacy). Add a comment: `// Departments are now dynamic — fetch from /api/departments`
- In `src/app/api/admin/import/route.ts`: Replace the hardcoded `DEPARTMENTS` array with a live Hasura query at the start of the POST handler
- In `src/app/(admin)/dashboard/admin/interns/page.tsx`: The form's department dropdown must be populated from a `GET /api/departments` call, not the constant
- Everywhere else that imports `DEPARTMENTS` from constants — replace with the API call

Part C — Department Management Page

Create `src/app/(admin)/dashboard/admin/departments/page.tsx`:

- Table showing: Department Name, Head (assigned mentor name or "None"), Number of Interns, Number of Mentors, Actions
- "Add Department" button → modal with name field + validation (Formik/Yup, Prompt 2 rules)
- Edit button → same modal pre-filled
- Delete button → confirm dialog (use existing `showConfirm`), blocked with a toast if department has users
- "Set as Head" dropdown per department: shows all mentors in that department. Selecting one sets `departments.head_id`. Only one head per department is enforced (the UI only shows a single select, the API enforces it).
- Add a link to this page in the admin Sidebar: label "Departments", icon `Building2` from Lucide, placed between "Mentors" and "Tasks"

PROMPT 5 — Extended Intern Profile Fields + Optional Field System

---

What to build

Interns should be creatable with minimal required info (admin fills required fields), and additional optional fields can be filled/edited later by the intern themselves and verified by mentor/admin.

Part A — Database migrations

Run these SQL statements (include them in your answer so the developer can apply them):

```sql
-- Add graduation_degree to interns table
ALTER TABLE interns ADD COLUMN graduation_degree TEXT;

-- Add recovery_email to users table
ALTER TABLE users ADD COLUMN recovery_email TEXT;

-- Add a verification status for intern extended profile
ALTER TABLE interns ADD COLUMN profile_verified BOOLEAN NOT NULL DEFAULT FALSE;
ALTER TABLE interns ADD COLUMN profile_verified_by UUID REFERENCES users(id) ON DELE
ALTER TABLE interns ADD COLUMN profile_verified_at TIMESTAMPTZ;
```

Part B — Required vs Optional fields

Required (admin must fill at creation):

- `name`, `email`, `department`, `mentorId`, `startDate`

Optional (filled by intern or admin later, verified by mentor/admin):

- `phone`, `endDate`, `collegeName`, `university`, `graduationDegree`

Part C — Intern self-service profile editing

In `src/app/profile/page.tsx`, when the logged-in user is an `intern`:

- Show an "Internship Details" section below the main profile

- Allow editing: `phone`, `collegeName`, `university`, `graduationDegree` (dropdown: `B.Tech`, `B.E.`, `MCA`, `BCA`, `M.Tech`, `MBA`, `B.Sc`, `M.Sc`, `Other`), `endDate`
- On save, set `profile_verified = false` (changes need re-verification)
- Show a status badge: `✓ Verified` (green) or `⏳ Pending Verification` (yellow)

Part D — Verification by mentor/admin

In the intern detail view (admin interns page and mentor interns page), add a "Verify Profile" button that:

- Sets `profile_verified = true`, `profile_verified_by = currentUserId`, `profile_verified_at = NOW()`
- Only visible if `profile_verified = false` and the viewer is admin or the intern's assigned mentor
- Show the verified-by name and date in a tooltip when hovering the verified badge

Part E — Dropdown fields

Replace any plain text input with a `<Select>` component where applicable:

- `department` → populated from `/api/departments` (dynamic)
- `graduationDegree` → static options list
- `status` → `active | completed | terminated | paused` (all four from the DB enum, not just `active | inactive`)
- `mentorId` → populated from mentors list filtered by selected department

---

PROMPT 6 — Filters on All Data Tables

What to build

The interns page already has URL-persisted filters (`name`, `department`, `mentorId`, `collegeName`). Implement the same pattern across **every table in the project**.

Tables that need filters

Page	Filter fields
Admin Interns	name, department, mentorId, status, startDate (range), collegeName ✓ already partial
Admin Mentors	name, department, status
Admin Tasks	title, priority, status, assignedToAll, deadlineFrom/To
Admin Reports	internId (search by name), reportDateFrom/To, mentorFeedback (has/hasn't)
Admin Attendance	userId (name search), date (range), status
Admin Logs	userId (name search), action, entityType, dateFrom/To
Mentor → My Interns	name, status
Mentor → Reports	internId (name search), reportDateFrom/To
Mentor → Tasks	title, priority, status
Intern → My Tasks	priority, status
Intern → Reports	reportDateFrom/To
Intern → Attendance	date range

Implementation pattern (follow the existing interns page pattern)

1. Filter state is stored in URL query params (use `useSearchParams` + `useRouter`)
2. Filters panel: a collapsible "Filters" row above the table, toggled by a `Filter` icon button (Lucide `SlidersHorizontal`)
3. Filter inputs: use `Input` and `Select` components already in the codebase
4. "Apply Filters" button: triggers a re-fetch
5. "Clear Filters" button: resets all filter params and re-fetches
6. Filter values are passed to the API route as query params; the API route passes them as Hasura `where` clauses
7. Active filter count badge: show on the filter toggle button (e.g., "Filters (3)")
8. Date range filters: two date `input[type=date]` fields labeled "From" / "To"

9. Filters persist on page reload (URL-based)

API side

For each API route, add `where` clause support in the Hasura query. Use Hasura's `_ilike` for text search, `_eq` for enums/IDs, `_gte`/`_lte` for date ranges. Add appropriate indexes to GraphQL queries in `src/lib/graphql/queries.ts`.

PROMPT 7 — Export Modal (All Tables)

What to build

A reusable `ExportModal` component that allows selective column/row export in multiple formats.

Component: `src/components/features/ExportModal.tsx`

Props:

```ts
interface ExportModalProps {
  isOpen: boolean;
  onClose: () => void;
  data: Record<string, unknown>[];        // full dataset (with filters already appli
  columns: { key: string; label: string }[]; // all available columns
  entityName: string;                     // e.g. "Interns", "Tasks"
  filename: string;                       // base filename without extension
}
```

Modal UI

1. **Header:** "Export [entityName]" with close button
2. **Column selector section** (left or top):
   - Checkboxes for each column, all checked by default
   - "Select All" / "Deselect All" toggle
3. **Row selector section:**

- Radio: "All rows (currently filtered)" vs "Selected rows only" (only shown if `selectedIds` are passed via prop)
- Show row count: "Exporting N rows"

4. **Preview table**: Small table showing the first 5 rows with only the selected columns (live updates as checkboxes change)
5. **Export buttons** (bottom bar):
   - `CSV` — use the existing `csv-utils.ts` helper, extend it if needed
   - `XLSX` — install and use `xlsx` package (`npm install xlsx`)
   - `PDF` — use `jspdf` + `jspdf-autotable` (`npm install jspdf jspdf-autotable`)
   - `HTML` — generate a self-contained styled HTML table string and trigger download via Blob

Integration

Wire up `ExportModal` in every table page. Replace the existing plain `downloadCSV` calls.

- Add an "Export" button (icon: `Download` from Lucide) to the top action bar of each table
- Pass the current **filtered** data to the modal (what's visible/fetched, not just the current page — fetch all pages for export)
- For the `selectedIds` feature: when rows are selected via the existing `BulkActionBar`, pass them so the user can choose "selected rows only"

PROMPT 8 — Improved Bulk Import with Error Report

What to build

Improve `BulkImportForm` to show a detailed, downloadable result after import.

Backend (`src/app/api/admin/import/route.ts`)

Change the per-row result object to include full detail:

```ts
```

```ts
interface RowResult {
  rowNumber: number;
  name: string;
  email: string;
  status: 'success' | 'failed' | 'skipped';
  reason?: string;   // human-readable error message
  fieldErrors?: Record<string, string>; // field-level errors
}
```

Return this in the response:

json

```
{
  "results": {
    "totalRows": 20,
    "success": 17,
    "failed": 2,
    "skipped": 1,
    "rows": [ ...RowResult[] ]
  }
}
```

Validation per row (add to import route):

- Required: `name`, `email`, `role`, `department`
- Email format check
- Department must exist in DB (dynamic check)
- Role must be `intern` or `mentor`
- For interns: `mentor_email` must be a valid existing mentor in the system
- Duplicate email → mark as `skipped` (not failed), reason: "User already exists"

Frontend (`src/components/features/BulkImportForm.tsx`)

After a successful import API call, show a **Results Panel**:

1. **Summary bar:** `✅ 17 Added  ❌ 2 Failed  ⏭ 1 Skipped` with color-coded chips
2. **Results table**: columns — Row #, Name, Email, Role, Status (badge), Reason
3. Filter tabs above the table: "All" | "Success" | "Failed" | "Skipped"
4. **Download Error Report button** (shown only if `failed > 0`):

- Generates a CSV with only the failed/skipped rows
- Columns match the original import template + a `"Error Reason"` column
- Pre-filled with the original data so the user can fix and re-upload

5. **Download Template button**: always visible, generates a blank CSV with the correct headers and one example row (commented)

PROMPT 9 — Real Notifications for Events & Announcements

What to build

Connect `NotificationBell` to real data. When a new event/announcement is created in the calendar, show a notification badge and live entry in the bell dropdown, filtered by the viewer's role.

Part A — Backend

1. Create `src/app/api/notifications/route.ts` (GET):
   - Returns all events created in the **last 7 days** that are relevant to the current user
   - "Relevant" means: `department_id IS NULL` (global) OR `department_id = user's department_id`
   - Also return events happening **today or tomorrow** (upcoming reminders)
   - Response shape:

```
json
```

```
{
  "notifications": [
    {
      "id": "event-uuid",
      "type": "event",
      "title": "New event: Team Meeting",
      "desc": "Today at 3:00 PM in Conference Room A",
      "time": "2h ago",
      "read": false,
      "eventId": "uuid"
    }
  ],
  "unreadCount": 3
}
```

- "Read" state: Store read status in `localStorage` keyed by `notifications_read_{userId}` (array of event IDs marked as read). Do not add a new DB table for read state — keep it simple.

2. Add a GraphQL query `GET_RECENT_EVENTS_FOR_USER` in `src/lib/graphql/queries.ts` that fetches events with `created_at >= NOW() - INTERVAL '7 days'` and filters by `department_id IS NULL OR department_id = $deptId`.

Part B — Frontend (`src/components/features/NotificationBell.tsx`)

1. Replace MOCK_NOTIFICATIONS with a real fetch from `/api/notifications` on mount and every 60 seconds (use `setInterval`, clear on unmount)
2. Add a `type: 'event'` case with a `Calendar` icon (already imported) in `getIcon()`
3. Clicking an event notification → navigate to `/dashboard/calendar`
4. Read state managed in `localStorage` (see Part A)
5. **Calendar icon badge in Sidebar**: In `src/components/layout/Sidebar.tsx`, add a small red dot badge on the "Calendar" nav item when there are unread event notifications. Use a shared context or a lightweight Zustand-style atom — or simply re-use the Redux `notificationSlice` by dispatching the unread count there.

Part C — Calendar page badge trigger

In `src/app/(shared)/dashboard/calendar/page.tsx`, when the page mounts, mark all current event notifications as read in localStorage.

PROMPT 10 — Email Verification for Email Changes (OTP)

What to build

When a user (any role) tries to change their email in `profile/page.tsx`, they must verify the **new email** via OTP before it is saved.

Flow

1. User types a new email in the email field and clicks "Update Email"
2. System sends a 6-digit OTP to the **new email address** (not the current one)
3. A modal opens: "We sent a code to `newemail@example.com`. Enter it below."
4. User enters the OTP → if correct, the email is updated in the DB and the session is refreshed
5. OTP expires after 10 minutes, max 3 attempts

Backend changes

`src/app/api/profile/route.ts` — split email update into two steps:

- **Step 1 —** `PATCH /api/profile` **with** `{ action: 'request_email_change', newEmail }`:
  - Validate new email format
  - Check new email is not already taken by another user
  - Generate a 6-digit OTP, store in `password_reset_tokens` table with `type = 'otp'`, `email = newEmail`, `expires_at = NOW() + INTERVAL '10 minutes'`
  - Send OTP email using the existing `emailService.ts` (add a new template `emailChangeOTP`)
  - Return `{ success: true, maskedEmail: "n***@example.com" }`
- **Step 2 —** `PATCH /api/profile` **with** `{ action: 'verify_email_change', newEmail, otp }`:
  - Look up the OTP in `password_reset_tokens`: match `email = newEmail`, `type = 'otp'`, `used_at IS NULL`, `expires_at > NOW()`
  - Check `attempts < 3`; if over, return 429 with message "Too many attempts. Request a new code."

- Increment `attempts` on each wrong attempt
- On success: update `users.email = newEmail`, mark OTP as used, trigger a NextAuth session update

Frontend (`src/app/profile/page.tsx`)

1. On "Update Email" button click → call Step 1 API
2. Open an OTP input modal (6-digit code, auto-focus, allow paste):
   - "Resend Code" link (enabled after 60 seconds countdown)
   - Error message under the input field for wrong OTP
   - Success: close modal, show toast "Email updated successfully", update displayed email
3. Use the existing `Modal` component from `src/components/ui/Modal.tsx`
4. OTP input: 6 separate single-character inputs (like a PIN entry) OR a single `<input maxLength={6}>` — your choice, but it must look polished

---

PROMPT 11 — Recovery Email & Basic Account Recovery

What to build

Allow users to add a recovery email. This is used as an **alternative delivery address** for password reset OTPs when the primary email is inaccessible.

> Scope note: Full 2FA (TOTP authenticator apps) is out of scope. A recovery email is sufficient and in-scope for an intern management system.

Part A — DB migration

```sql
-- Already added in Prompt 5. Confirm it exists:
ALTER TABLE users ADD COLUMN IF NOT EXISTS recovery_email TEXT;
```

Part B — Profile page UI

In `src/app/profile/page.tsx`, add a "Recovery Email" section:

- Text input for recovery email
- Same OTP verification flow as Prompt 10 (verify the recovery email address before saving)
- Show current recovery email masked: `r****@gmail.com` — with a "Remove" button
- Badge: `✓ Verified` once set

Part C — Password reset flow enhancement

In `src/app/api/auth/forgot-password/route.ts`:

- After the current logic that sends OTP to primary email, also check if `users.recovery_email` exists
- If the primary email lookup fails (user not found by that email), try finding the user by `recovery_email`
- In the "we sent a code" confirmation page / UI, add: *"Didn't receive it? We can also send to your recovery email."* → a secondary "Send to recovery email" button that re-sends to the recovery address

In `src/app/auth/forgot-password/page.tsx`:

- Add an "I can't access this email" link below the OTP entry
- On click, if the user has a recovery email → re-send to recovery email (masked display)

PROMPT 12 — Dark Mode Improvements

What to audit and fix

The project uses CSS custom properties for theming (visible in `globals.css`). The dark mode exists but has known issues.

Specific fixes required

1. **SweetAlert2 dialogs** (`src/lib/notifications.ts`):
   - `showToast`, `showAlert`, `showConfirm` currently hardcode `background: '#ffffff'` and `color: '#1e293b'`

- Fix: detect dark mode using `window.matchMedia('(prefers-color-scheme: dark)')` OR by checking if the `<html>` element has a `dark` class (check how `ThemeProvider.tsx` sets the theme)
- In dark mode, use: `background: 'var(--surface-overlay)'` → resolve to its actual hex or use `#1e293b` / `#0f172a` as fallback
- SweetAlert2 doesn't support CSS variables in its config — resolve the var at call time using `getComputedStyle(document.documentElement).getPropertyValue('--surface-overlay').trim()`

2. **`PerformanceChart`** (`src/components/features/PerformanceChart.tsx`):
   - The Recharts `<Tooltip>` `contentStyle.backgroundColor` uses `'var(--surface-overlay)'` — Recharts renders in SVG/inline styles and cannot read CSS variables
   - Fix: use the same `getComputedStyle` trick to resolve the value at render time via a `useMemo` that depends on a theme-change event
3. **Login / Auth pages** (`src/app/auth/login/page.tsx`, `forgot-password/page.tsx`, etc.):
   - Check that background and card colors use the CSS variable system, not hardcoded hex
   - Fix any hardcoded light colors found
4. **`globals.css` audit**:
   - Ensure every `:root` color variable has a corresponding `[data-theme="dark"]` or `.dark` override
   - List any missing dark overrides you find and add them
5. **Theme toggle in `DashboardHeader.tsx`**:
   - If no theme toggle button exists, add one: Sun/Moon icon (`Sun` / `Moon` from Lucide), clicking it toggles the `dark` class on `<html>` and saves preference to `localStorage`
   - Respect `prefers-color-scheme` as the initial default if no stored preference

---

PROMPT 13 — Analytics Dashboard (Recharts, In-App)

Context on Apache Superset

Apache Superset is a standalone BI server that requires its own Docker deployment and is far outside the scope of this project. Instead, **upgrade the existing Recharts-based `PerformanceChart` component** into a proper, data-connected analytics section inside the admin dashboard. This is the appropriate and in-scope approach.

What to build

Create a new `src/components/features/AnalyticsDashboard.tsx` component used in `src/app/(admin)/dashboard/admin/page.tsx`.

Charts to include (all using real API data)

1. Intern Status Breakdown — Donut/Pie chart

- Data from: count of interns grouped by `status` (active/completed/terminated/paused)
- API: extend `GET /api/admin/stats` to return `internsByStatus`

2. Task Completion Rate Over Time — Area/Line chart

- X-axis: last 14 days
- Y-axis: number of tasks completed per day
- Data: query `task_assignments` where `status = 'completed'`, group by `date(assigned_at)`
- API: add `taskCompletionTimeline` to `GET /api/admin/stats`

3. Attendance Rate by Department — Horizontal Bar chart

- X-axis: % attendance rate
- Y-axis: department names
- Data: `attendance` table, `status = 'present'`, grouped by department
- API: add `attendanceByDepartment` to stats

4. Hours Worked per Intern — Vertical Bar chart (top 10 interns by hours)

- Data: `SUM(hours_worked)` from `reports`, grouped by `intern_id`, join to profile for name

- API: add `topInternsByHours` to stats

5. Reports Submitted per Day — Sparkline/small area chart

- Last 30 days, count of reports per day
- Shown as a small chart, not full-size

UI layout

- 2×2 grid of chart cards on desktop, stacked on mobile
- Each card has: title, subtitle ("Last 14 days"), the chart, and a small data table toggle (clicking "View Data" shows the raw numbers in a compact table below the chart)
- Loading skeletons while fetching (use existing `Skeleton` component)
- Empty state illustration when no data exists

Recharts implementation notes

- Always wrap charts in `<ResponsiveContainer width="100%" height={300}>`
- Use CSS variable color resolution trick from Prompt 12 for theme-aware colors
- Use `recharts` `CustomTooltip` pattern (already exists in `PerformanceChart.tsx`) for consistent tooltip styling
- Color palette: use the 5-color array already in `PerformanceChart.tsx`: `var(--color-primary)`, `var(--color-info)`, `var(--color-warning)`, `var(--color-error)`, `var(--color-success)`

PROMPT 14 — Rate Limiting & Basic DDoS Protection

Scope

Full DDoS mitigation (CDN-level, Cloudflare, etc.) is infrastructure-level and out of scope for the application codebase. What we CAN do in Next.js:

- Rate limiting on sensitive API routes (auth, OTP, password reset)
- Brute-force protection on the login endpoint
- Lockout after repeated failed OTP attempts (already partially done via `attempts` column)

Implementation

Install: `npm install @upstash/ratelimit @upstash/redis` — BUT since this project may not have Upstash, use a **simple in-memory rate limiter** that works without external dependencies (acceptable for a single-instance deployment):

Create `src/lib/rateLimit.ts`:

```ts
// Simple in-memory rate limiter using a Map
// Resets on server restart — acceptable for intern management scale
export function createRateLimiter(options: {
  windowMs: number;   // time window in ms
  max: number;        // max requests per window per key
}): (key: string) => { allowed: boolean; remaining: number; resetAt: number }
```

Apply rate limiting in:

1. `/api/auth/[...nextauth]` **(login)**: 10 attempts per IP per 15 minutes. On block, return 429.
    - Middleware cannot intercept NextAuth's internal handler, so add rate limiting inside `authorize()` in `src/lib/auth.ts` using the request IP from headers
2. `/api/auth/forgot-password`: 3 requests per email per 10 minutes
3. `/api/auth/verify-otp`: 5 attempts per email per 10 minutes (the DB `attempts` column handles per-token, this handles flooding)
4. `/api/profile` **(email change OTP)**: 3 requests per userId per 10 minutes
5. **In** `src/middleware.ts` (from Prompt 1): Add a blanket limit of 100 requests per IP per minute across all API routes. If exceeded, return a JSON 429 response. Use the `x-forwarded-for` header as IP key.

UI handling

When a 429 is received on the frontend, show a toast: *"Too many attempts. Please wait X minutes before trying again."* — parse the `Retry-After` header if present, otherwise say "a few minutes".

END OF PROMPTS

Recommended execution order

Order	Prompt	Why
1st	Prompt 1 (Middleware)	Foundation — everything depends on proper auth
2nd	Prompt 4 (Departments)	Removes hardcoding; subsequent prompts use dynamic depts
3rd	Prompt 2 (Formik/Yup)	Validation foundation for all form-related prompts
4th	Prompt 5 (Extended Fields)	Schema changes; do before export/filter
5th	Prompt 6 (Filters)	All tables now have proper filterable data
6th	Prompt 7 (Export)	Filters must exist before export makes sense
7th	Prompt 8 (Bulk Import)	Standalone improvement
8th	Prompt 3 (Password Breach)	Depends on Prompt 2 Yup schemas
9th	Prompt 9 (Notifications)	Depends on real event data being in place
10th	Prompt 10 (Email OTP)	Depends on existing OTP infrastructure
11th	Prompt 11 (Recovery Email)	Depends on Prompt 10 pattern
12th	Prompt 12 (Dark Mode)	UI polish — do after logic is stable
13th	Prompt 13 (Analytics)	Depends on clean data + stable API
14th	Prompt 14 (Rate Limiting)	Security hardening — do last

PROMPT 15 — Full-System Consistency Audit & Cleanup

> Run this prompt **last**, after all previous prompts (1–14) are complete. Your job is a senior developer doing a final sweep before handoff. Do not add new features. Fix what is broken, inconsistent, or duplicated.

SECTION A — Frontend: Component Usage Audit

The project has a complete Design System v2.0 component library in `src/components/ui/`. The rule is: **if a UI primitive exists, use it — never write raw HTML equivalents inline.** Audit and fix every violation below.

A1. `Badge` component — replace all inline badge spans

Scan every `.tsx` file in `src/app/` for patterns like:

```tsx
<span className="badge badge-success">...</span>
<span className="badge-warning">...</span>
```

Replace every occurrence with the `<Badge>` component:

```tsx
import { Badge } from '@/components/ui/Badge';
<Badge variant="success">Active</Badge>
```

The `Badge` variants are: `success | warning | error | info | primary | neutral`. Map status values consistently:

- intern/task `active` → `success`
- intern/task `pending` → `warning`
- intern/task `in-progress` → `info`
- intern/task `completed` → `success`
- intern/task `review` → `warning`
- intern `terminated` → `error`

- intern `paused` → `neutral`
- priority `high` → `error`, `medium` → `warning`, `low` → `neutral`

This mapping must be **identical across all pages** (admin interns, mentor interns, task lists, etc.).

A2. `TextArea` component — replace all raw `<textarea>` elements

Scan every form modal in `src/app/` for raw `<textarea>` tags. Replace with:

```tsx
import { TextArea } from '@/components/ui/TextArea';
<TextArea label="Description" error={errors.description} {...formik.getFieldProps('d
```

The `TextArea` component already handles label, `form-error`, and `form-hint` — do not duplicate those elements alongside it.

A3. `Button` component — replace inline button loading patterns

Scan for patterns like:

```tsx
<button disabled={loading}>{loading ? <Loader2 className="animate-spin" /> : null}
```

Replace with:

```tsx
<Button loading={loading} type="submit">Submit</Button>
```

The `Button` component already has a built-in `loading` prop with a spinner. Do not import `Loader2` for submit buttons — it creates two visual spinner styles in the same project.

A4. `Card` component — replace inline `<div className="card">` blocks

Scan for `<div className="card p-6">`, `<div className="card-stat">` etc. used directly without the `Card`/`StatCard` component wrapper. Replace with:

```tsx
<Card padding="md">...</Card>
<StatCard label="Total Interns" value={42} icon={<Users />} />
```

Exception: chart containers and table containers may keep the raw `.card` class if they have complex internal layout that the component wrapper would complicate.

A5. `StatsGrid` — ensure all stat grids use it

Every dashboard page that renders 3–5 `StatCard`s in a row must wrap them in `<StatsGrid stats={[...]} loading={loading} />` instead of mapping `StatCard` manually. This ensures the `.stats-grid` responsive CSS grid is applied consistently. Check:
`src/app/(admin)/dashboard/admin/page.tsx`,
`src/app/(mentor)/dashboard/mentor/page.tsx`,
`src/app/(intern)/dashboard/intern/page.tsx`.

A6. `Pagination` component — fix internal raw `<select>`

Inside `src/components/ui/Pagination.tsx`, the page-size picker uses a raw `<select className="select">` tag. Replace it with the `Select` component:

```tsx
import { Select } from './Select';
<Select value={String(pageSize)} onChange={(e) => onPageSizeChange(Number(e.target.v
  options={[
    { value: '5', label: '5 per page' },
    { value: '10', label: '10 per page' },
    { value: '25', label: '25 per page' },
    { value: '50', label: '50 per page' },
  ]}
/>
```

A7. `Select` component — replace all raw `<select>` elements in forms

Scan all form modals and filter panels for raw `<select>` tags. Replace with the `Select` component so field-level error display is consistent with `Input` and `TextArea`.

SECTION B — Layout & Chrome Inconsistencies

B1. Fix the double mobile overlay bug

Problem: Both `src/components/layout/Sidebar.tsx` and `src/components/layout/DashboardLayout.tsx` independently render a `.sidebar-overlay` div when on mobile. This results in two overlapping backdrops.

Fix: Remove the overlay div from `Sidebar.tsx` entirely. Keep only the one in `DashboardLayout.tsx` which already has the `handleOverlayClick` handler. The Sidebar's

open/close is controlled by Redux state (`isCollapsed`) — it doesn't need to own its own overlay.

B2. Add `NotificationBell` to `DashboardHeader`

Problem: `src/components/features/NotificationBell.tsx` is a fully built component but is **never rendered anywhere** — it is not imported in `DashboardHeader.tsx`.

Fix: In `src/components/layout/DashboardHeader.tsx`, import and render `NotificationBell` in the right-side actions bar, between the theme toggle and the user avatar:

```tsx
import { NotificationBell } from '../features/NotificationBell';
// In the JSX right-side actions div:
<NotificationBell />
```

It should appear for all roles (admin, mentor, intern), not just admin.

B3. Fix the mentor Sidebar linking to an admin route

Problem: In `src/components/layout/Sidebar.tsx`, the mentor navigation array contains:

```ts
{ href: "/dashboard/admin/attendance", label: "Attendance Monitor", ... }
```

This is an admin-only route. After Prompt 1 (middleware), a mentor visiting this URL will be redirected to `/access-denied`.

Fix: Change this link to `/dashboard/mentor/attendance` if that page exists, or remove it from the mentor nav and link to the shared equivalent. Check whether `src/app/(mentor)/dashboard/mentor/attendance/` exists — if not, create a basic mentor attendance view page that shows attendance for the mentor's assigned interns only, using the existing `AttendanceTable` component.

B4. Remove `DashboardLayout` auth redirect after Prompt 1

Problem: `src/components/layout/DashboardLayout.tsx` contains:

```ts
if (status === "unauthenticated") { router.push("/auth/login"); }
```

After Prompt 1 (middleware) is implemented, the middleware handles this server-side before the component even renders. The client-side redirect in `DashboardLayout` becomes redundant and can cause a brief flash (renders null → redirects).

Fix: Remove the `router.push('/auth/login')` and the unauthenticated check from `DashboardLayout`. Keep the loading spinner and the `if (!mounted || status === 'loading') return <spinner>` logic. The `if (status === 'unauthenticated') return null` guard can stay as a safety net but should not redirect.

B5. Fix `SearchHeader` using wrong CSS class

Problem: `src/components/features/SearchHeader.tsx` wraps its content in `<div className="page-header mb-6">`. The `.page-header` class in `globals.css` is designed for the **sticky top navigation bar** (`DashboardHeader`). Using it inside page content creates a visual conflict — it may have `position: sticky`, `top: 0`, `z-index`, or padding that is wrong when nested inside `.page-content`.

Fix: In `SearchHeader.tsx`, replace `className="page-header mb-6"` with `className="mb-6"`. The inner layout (`flex justify-between`, the colored left-border accent bar, action buttons) is already correct — only the outer wrapper class is wrong. Verify this visually — the page title section should sit flush inside the content area, not behave like a second sticky header.

B6. Extract the brand logo into a shared component

Problem: The brand identity (indigo gradient square + `Building` icon + "Intern Management" text) is copy-pasted in both `Sidebar.tsx` and `DashboardHeader.tsx` with identical inline classes. Any future rebrand touches two files.

Fix: Create `src/components/ui/BrandLogo.tsx`:

```tsx
interface BrandLogoProps {
  collapsed?: boolean; // shows icon only when true
}
export function BrandLogo({ collapsed = false }: BrandLogoProps) { ... }
```

Replace both occurrences in `Sidebar.tsx` and `DashboardHeader.tsx` with `<BrandLogo collapsed={isCollapsed} />`.

SECTION C — Backend: API Route Consistency Audit

C1. Standardise all API routes to use `responseHelper.ts`

Problem: `src/lib/api/responseHelper.ts` exports `successResponse()`, `errorResponse()`, `unauthorized()`, `forbidden()`, `notFound()`, `validationError()`. These exist specifically so every API route returns the **same JSON shape**. However, most routes call

`NextResponse.json()` directly with ad-hoc shapes like `{ error: "..." }`, `{ message: "..." }`, `{ data: [...] }`, `{ interns: [...] }`.

Fix: Audit every file in `src/app/api/`. For each route:

- Replace `NextResponse.json({ error: "Unauthorized" }, { status: 401 })` → `unauthorized()`
- Replace `NextResponse.json({ error: "Forbidden" }, { status: 403 })` → `forbidden()`
- Replace `NextResponse.json({ error: "Not found" }, { status: 404 })` → `notFound()`
- Replace `NextResponse.json({ interns: [...] })` → `successResponse({ interns: [...] })`
- Replace `NextResponse.json({ error: "..." }, { status: 400/422 })` → `errorResponse("...", ERROR_CODE, status)`

All frontend `fetch()` calls that currently destructure `data.interns`, `data.users`, etc. must be updated to `data.data.interns`, `data.data.users` after this change — **or** keep the existing API response shapes and have `successResponse` wrap them inside `data`. Choose one approach and apply it uniformly. **Do not mix both shapes.**

C2. Standardise auth checks to use `authGuard.ts`

Problem: `src/lib/api/authGuard.ts` exports `getAuthUser()` and `requireRole()`. These are convenience wrappers around `getServerSession`. However, API routes bypass this and call `auth()` (or `getServerSession`) directly, then manually check `session.user.role`.

Fix: In every API route, replace:

```ts
const session = await auth();
if (!session || session.user.role !== 'admin') {
  return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
}
```

With:

```ts
```

```
import { requireRole } from '@/lib/api/authGuard';
const { user, response } = await requireRole(['admin']);
if (response) return response; // returns 401/403 automatically
```

This ensures auth error responses are always consistent and the role check is never forgotten.

C3. Enforce consistent activity logging

Problem: `src/lib/activityService.ts` exports `logActivity()`. Not all mutation routes (POST/PATCH/DELETE) call it, making the `activity_logs` table incomplete and the admin logs page unreliable.

Fix: Audit every POST, PATCH, DELETE API route. Add a `logActivity()` call after every successful mutation. Use a consistent `action` naming convention:

Operation	`action` value	`entityType`
Create intern	`CREATE_INTERN`	`intern`
Update intern	`UPDATE_INTERN`	`intern`
Delete intern	`DELETE_INTERN`	`intern`
Create mentor	`CREATE_MENTOR`	`mentor`
Update mentor	`UPDATE_MENTOR`	`mentor`
Delete mentor	`DELETE_MENTOR`	`mentor`
Create task	`CREATE_TASK`	`task`
Assign task	`ASSIGN_TASK`	`task`
Update task status	`UPDATE_TASK_STATUS`	`task`
Submit report	`SUBMIT_REPORT`	`report`
Update report feedback	`UPDATE_REPORT_FEEDBACK`	`report`
Clock in	`CLOCK_IN`	`attendance`
Clock out	`CLOCK_OUT`	`attendance`
Create event	`CREATE_EVENT`	`event`
Bulk import	`BULK_IMPORT`	`import`
Change password	`CHANGE_PASSWORD`	`user`
Change email	`CHANGE_EMAIL`	`user`

The `logActivity` call must be **fire-and-forget** (already is — the service swallows errors). Never `await` it in a way that blocks the response.

C4. Consistent HTTP status codes

Audit all API routes for these specific corrections:

- Creating a resource successfully → `201 Created` (not `200 OK`)

- Successful deletion → `200 OK` with `{ success: true }` (not `204 No Content` — our frontend checks the body)
- Validation errors → `422 Unprocessable Entity` (not `400 Bad Request`)
- Auth not present → `401 Unauthorized`
- Auth present but wrong role → `403 Forbidden`
- Resource not found → `404 Not Found`
- Duplicate/conflict (e.g., email already exists, duplicate department name) → `409 Conflict`

SECTION D — Global Search Consistency

D1. Verify `/api/search` route correctness

`src/app/api/search/` exists. Verify its route handles:

- Query param: `q` (string, min 2 chars — reject shorter queries with `{ results: [] }`)
- Searches: intern profiles (by name, email), mentors (by name), tasks (by title)
- Role-scoped results: if the searcher is a `mentor`, return only their assigned interns' results. If `intern`, return only their own tasks. If `admin`, return everything.
- Response shape: `{ results: [{ id, name/title, type: 'intern'|'mentor'|'task', subtitle: string }] }`
- The `DashboardHeader` search renders the result's `name || title` and `type`. Make sure the search API returns both fields.

D2. Fix: global search only shown for admin

In `DashboardHeader.tsx`, the search bar is conditionally rendered only for `role === 'admin'`. But mentors also benefit from searching their interns and tasks.

Fix: Show the search bar for both `admin` and `mentor`. The `/api/search` route already scopes results by role (after D1 fix). The intern role does not need global search (they only see their own data).

SECTION E — Design & Visual Consistency

E1. SweetAlert2 dark-mode awareness (summary fix from Prompt 12)

After Prompt 12, verify this pattern is applied in `src/lib/notifications.ts` for all three functions (`showToast`, `showAlert`, `showConfirm`):

```ts
function getSwalTheme() {
  const isDark = document.documentElement.classList.contains('dark');
  return {
    background: isDark ? '#1e293b' : '#ffffff',
    color: isDark ? '#f1f5f9' : '#1e293b',
  };
}
```

Call `getSwalTheme()` inside each function before the `Swal.fire()` call. Verify the three SweetAlert2 functions are the **only place** Swal is called in the project — scan for any stray `Swal.fire()` calls in page files and replace them with the `showToast`/`showAlert`/`showConfirm` helpers.

E2. Status badge colour consistency check

Verify the Badge variant mapping from Section A1 is applied in all of the following files and that no file uses a different colour for the same status:

- `src/app/(admin)/dashboard/admin/interns/page.tsx`
- `src/app/(admin)/dashboard/admin/mentors/page.tsx`
- `src/app/(admin)/dashboard/admin/tasks/page.tsx`
- `src/app/(mentor)/dashboard/mentor/interns/page.tsx`
- `src/app/(mentor)/dashboard/mentor/tasks/page.tsx`
- `src/app/(intern)/dashboard/intern/tasks/page.tsx`

Create a shared utility function to ensure this:

```ts
```

```tsx
// src/lib/utils/statusColors.ts
export function getStatusBadgeVariant(status: string): BadgeVariant {
  const map: Record<string, BadgeVariant> = {
    active: 'success', completed: 'success',
    'in-progress': 'info', review: 'warning',
    pending: 'warning', terminated: 'error',
    paused: 'neutral',
  };
  return map[status] ?? 'neutral';
}

export function getPriorityBadgeVariant(priority: string): BadgeVariant {
  return priority === 'high' ? 'error' : priority === 'medium' ? 'warning' : 'neutra
}
```

Import and use this in every page instead of inline ternary chains.

E3. Table structure consistency

Every data table in the project must follow the same structure:

1. Page title section (using `SearchHeader` with page title and primary action button)
2. Filter row (collapsible, from Prompt 6)
3. `BulkActionBar` (appears when rows selected)
4. Table with `<thead>` / `<tbody>`, sortable column headers, row checkboxes
5. Empty state (when no data) — a centred message with a relevant Lucide icon
6. `Pagination` at the bottom

Audit every table page. Fix any that skip steps or implement them differently (e.g., some pages may have the "Add" button in a different position, some may not have empty states).

Empty state pattern — create `src/components/ui/EmptyState.tsx`:

tsx

```tsx
interface EmptyStateProps {
  icon: React.ReactNode;
  title: string;
  description: string;
  action?: React.ReactNode;
}
```

Use it in every table's empty state. Current pages likely have ad-hoc empty messages.

E4. Loading skeleton consistency

The `Skeleton` component (`src/components/ui/Skeleton.tsx`) and `StatCardSkeleton` exist. Verify every page that fetches data:

- Shows skeletons **while loading** (not a spinner or nothing)
- Skeletons must match the shape of the content (table skeletons for tables, card skeletons for stat grids)
- The `StatsGrid` already handles its own loading state — verify it's passed `loading={loading}` correctly

---

SECTION F — Final Checklist

After making all the above fixes, run through this checklist and confirm each item is resolved:

Component reuse

- [ ] No raw `<textarea>`, `<select>`, or `<input>` elements inside form modals (all use `Input`, `Select`, `TextArea`)
- [ ] No inline `<span className="badge ...">` — all use `<Badge>`
- [ ] No inline `<button disabled={loading}>` for submit actions — all use `<Button loading>`
- [ ] `BrandLogo` component used in both `Sidebar` and `DashboardHeader`
- [ ] `EmptyState` component used in all table empty states
- [ ] `getStatusBadgeVariant()` and `getPriorityBadgeVariant()` used in all table pages
- [ ] `StatsGrid` wraps all stat card groups

Layout

- [ ] Only one `.sidebar-overlay` div rendered at a time (fixed double overlay)
- [ ] `NotificationBell` visible in `DashboardHeader` for all roles
- [ ] Mentor sidebar links to a valid, accessible route for attendance
- [ ] `SearchHeader` uses correct non-sticky wrapper class
- [ ] Global search visible for `admin` and `mentor` roles

Backend

- [ ] All API routes use `responseHelper.ts` functions — no raw `NextResponse.json()`
- [ ] All API routes use `authGuard.ts` `requireRole()` — no inline role checks
- [ ] All POST/PATCH/DELETE routes call `logActivity()` on success

- [ ] HTTP status codes follow the convention in Section C4
- [ ] `/api/search` returns role-scoped results with the correct shape

Notifications

- [ ] `showToast` / `showAlert` / `showConfirm` are the only Swal entry points — no stray `Swal.fire()` calls
- [ ] All three SweetAlert2 helpers are dark-mode aware